

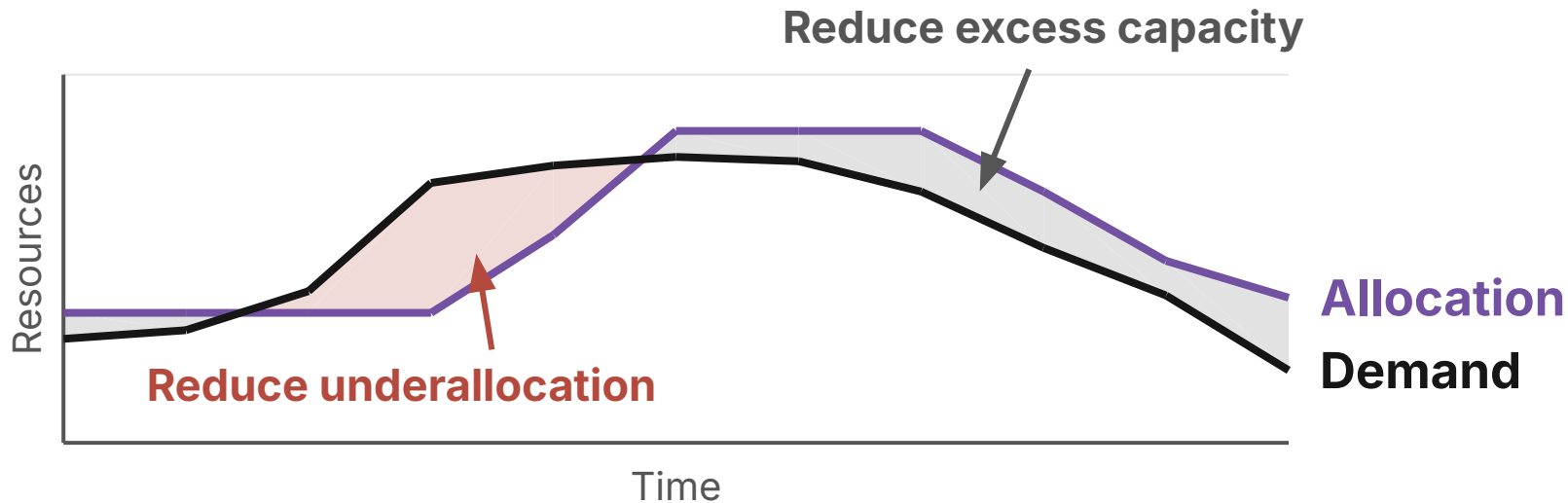


Autoscaling is not ^{just} resource allocation

Sacheendra Talluri, Marvin Beckers, Frank Rosner, Ashwath Singh, Jayme Bird, Francesco Ciocchetti, Amit Joshi, Manas Alekar, Manish Gill, and other ClickHouse Engineers

HPTS 2026

My initial goal: match resources to demand

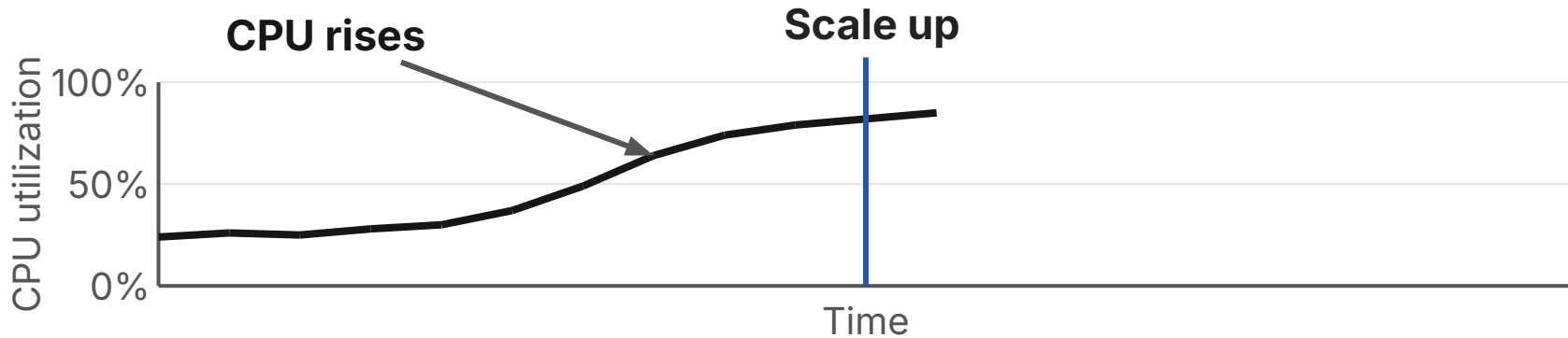


Minimize the demand-allocation gap.

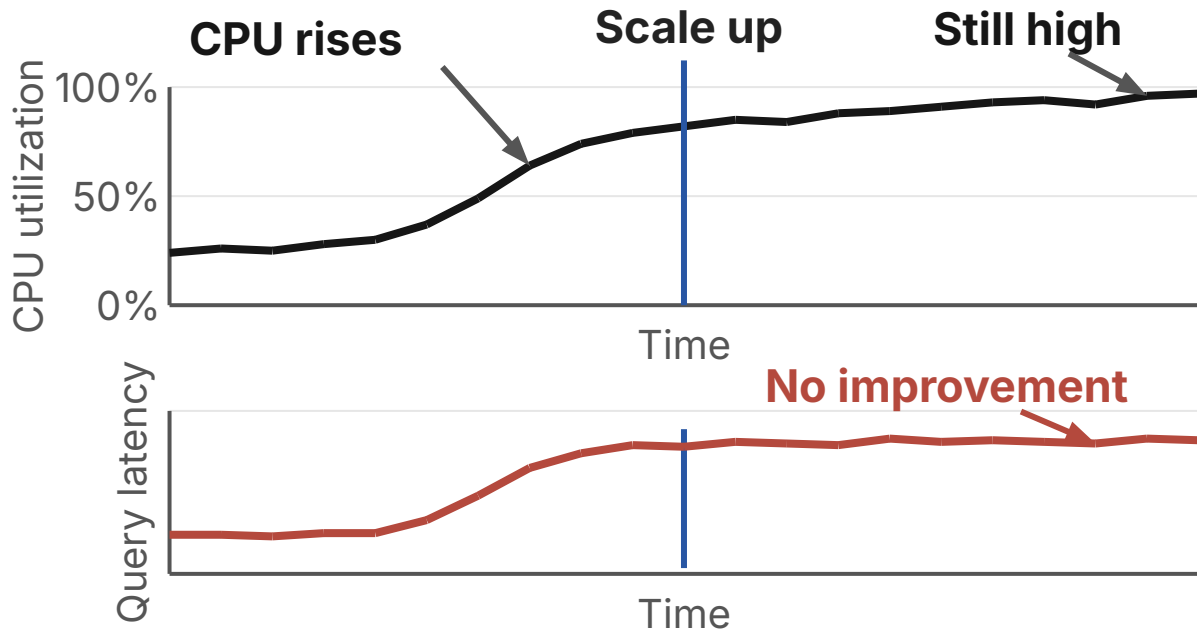
The autoscaler saw high CPU utilization



The autoscaler scaled up



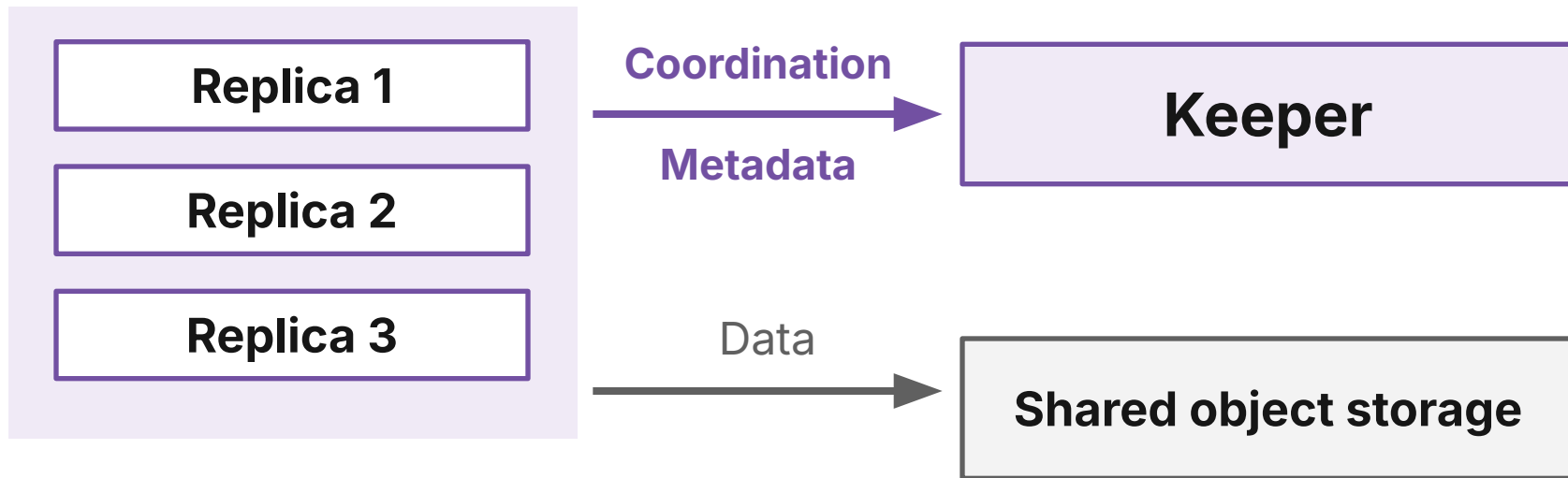
CPU stayed high. Latency did not improve



Dex Ezekiel on Unsplash

Replicas share a coordination service

ClickHouse replicas



More replicas mean more Keeper clients.

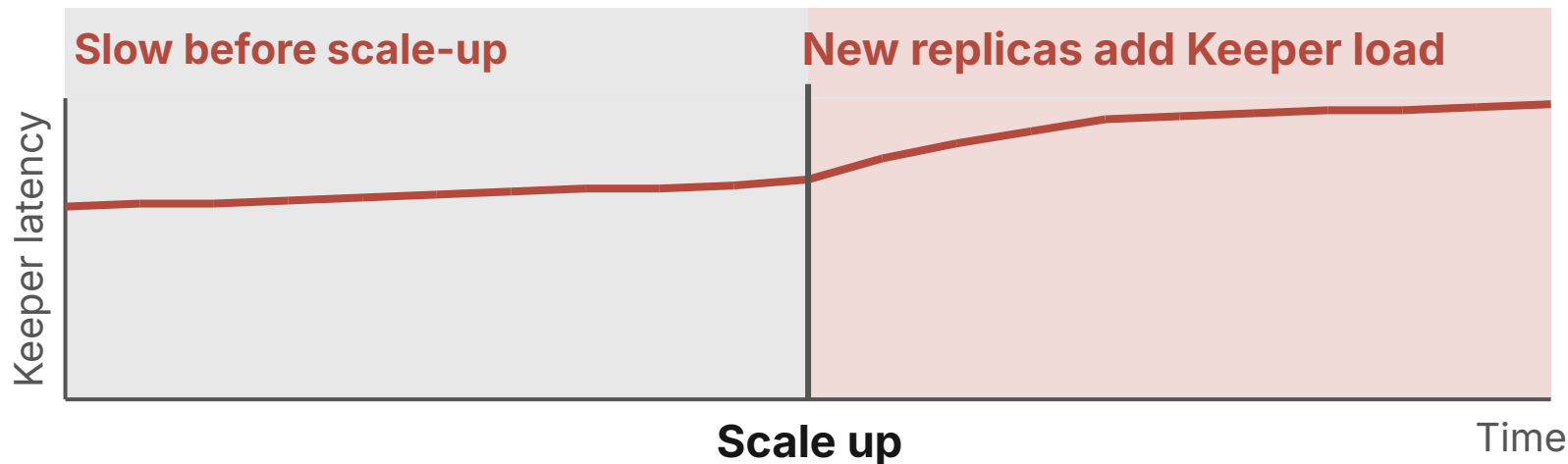
Keeper was already the bottleneck

Slow Keeper responses

→ Merges slow

→ Parts pile up

→ Queries slow,
CPU rises



Merges: background compaction

Check Keeper health before adding replicas

DEPENDENCY HEALTH

Keeper is unhealthy

SCALING SAFEGUARD

Throttle new replicas

EFFECT

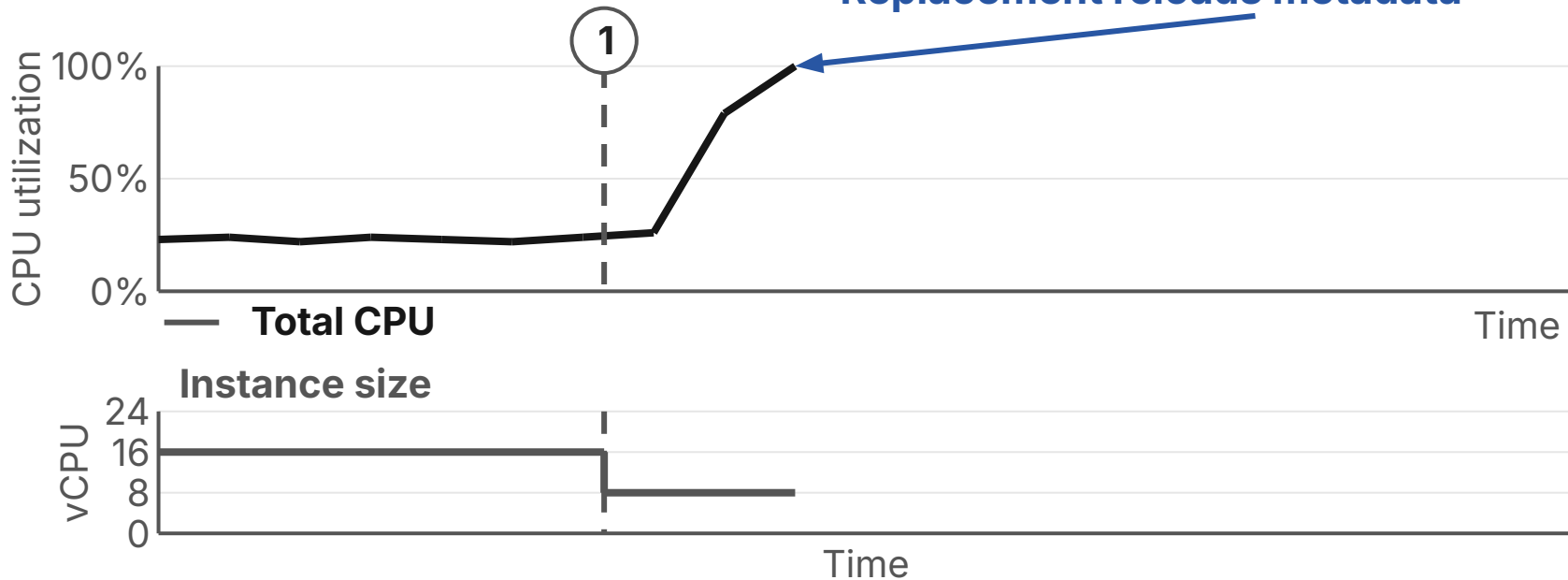
Avoid further Keeper overload

The underlying slowdown still needs a fix.

Metadata loading used all available CPUs

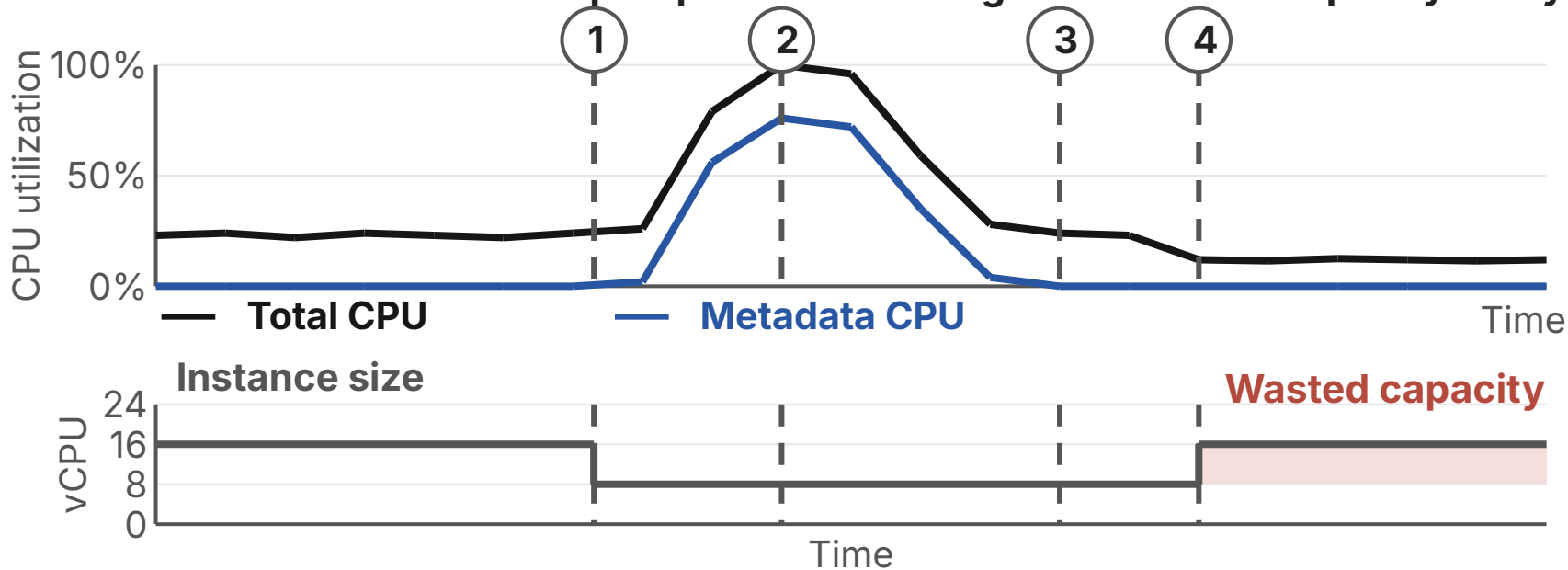
1 Scale down

Replacement reloads metadata



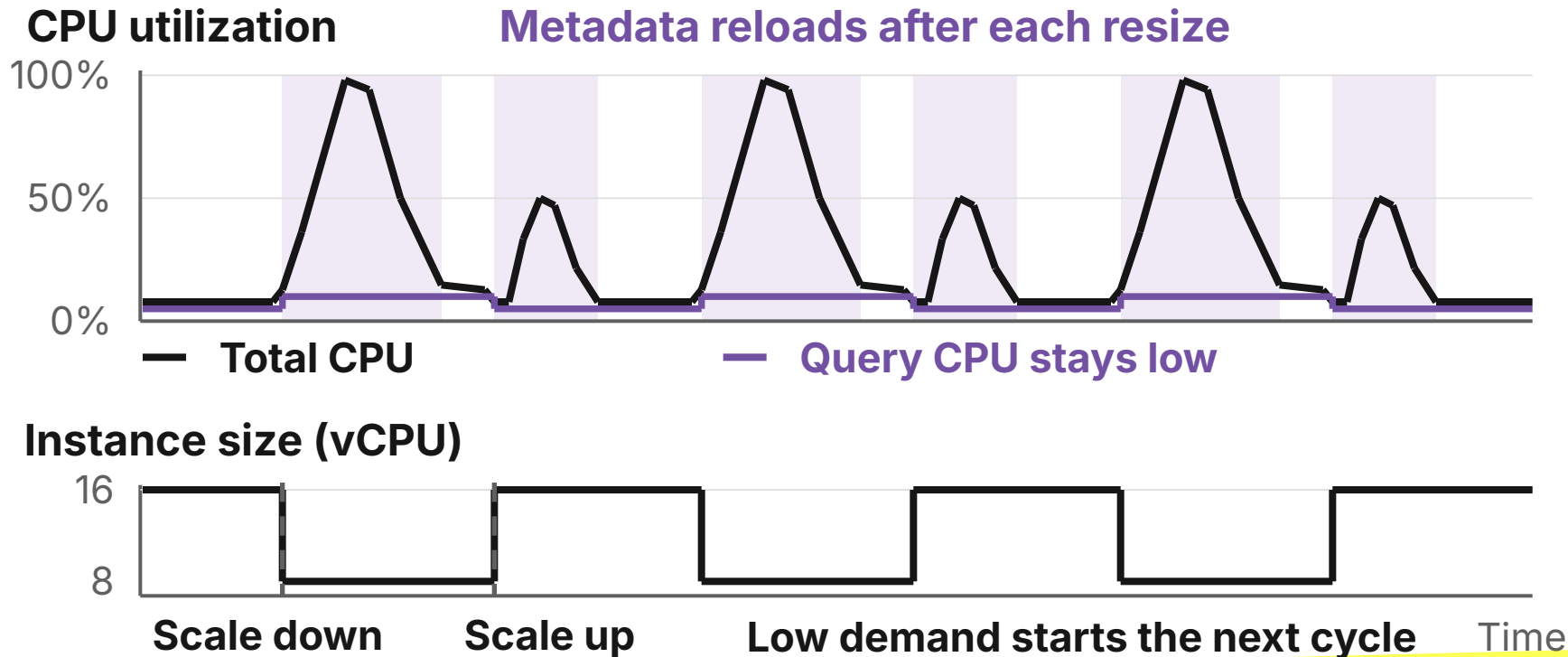
Metadata loading triggered a scale-up

1 Scale down 2 Scale-up request 3 Loading done 4 Capacity ready

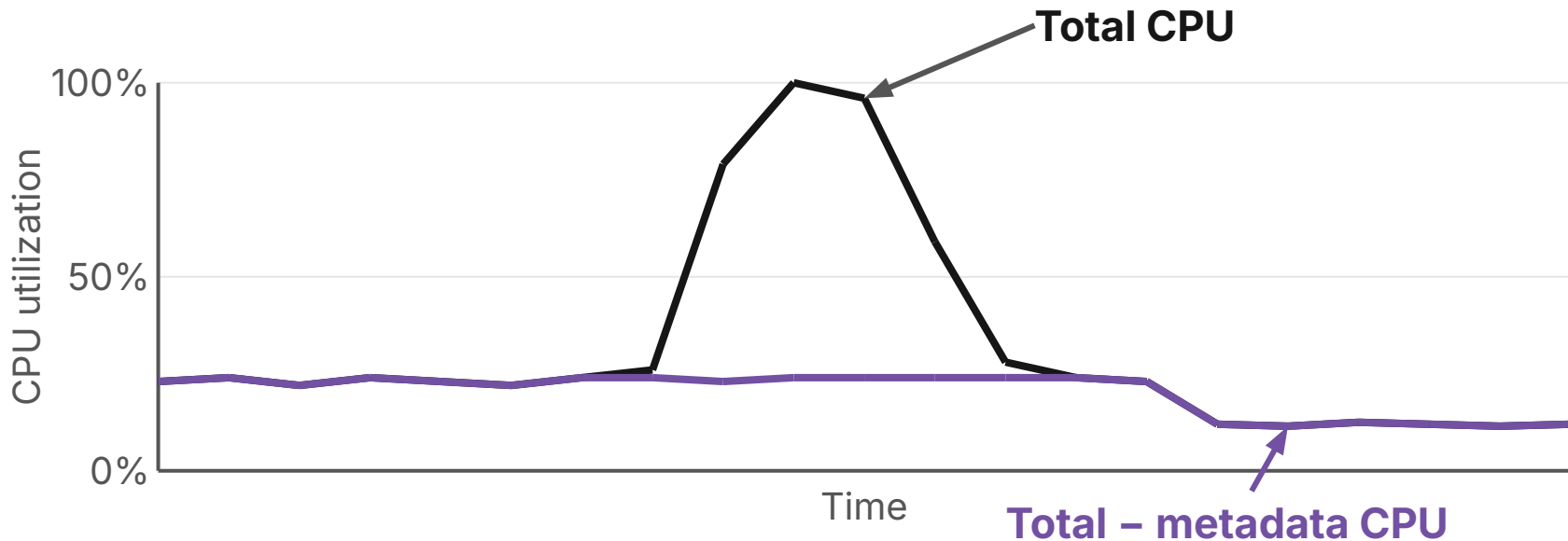


The extra capacity is wasted

Not just wasted capacity. Oscillation



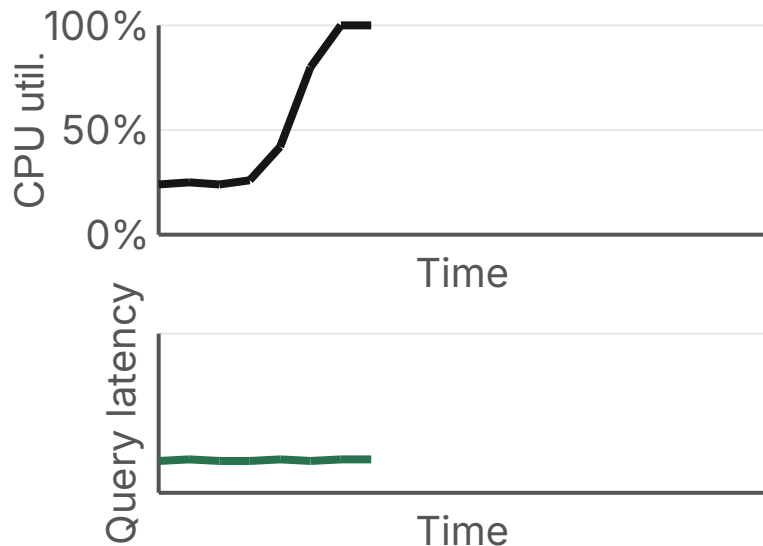
We excluded metadata-loading CPU



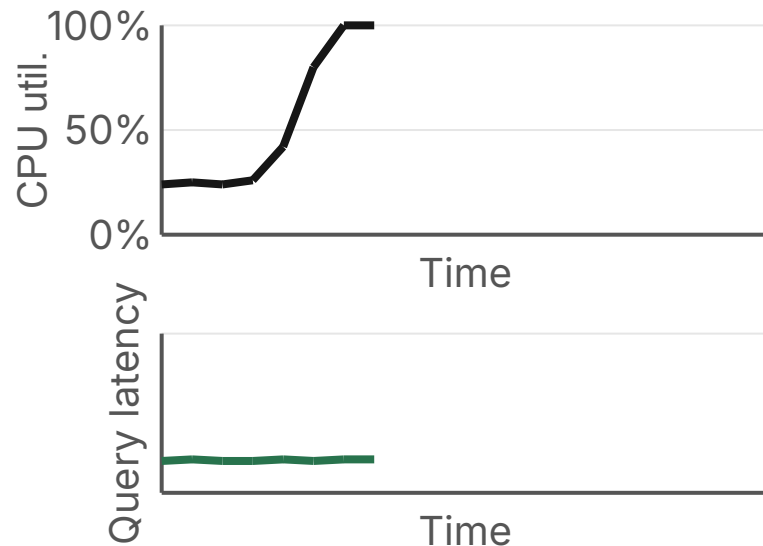
But can we exclude other background work the same way?

Both merges reach 100% CPU

Merge workload A



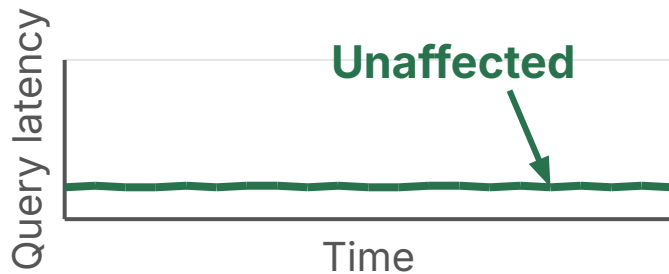
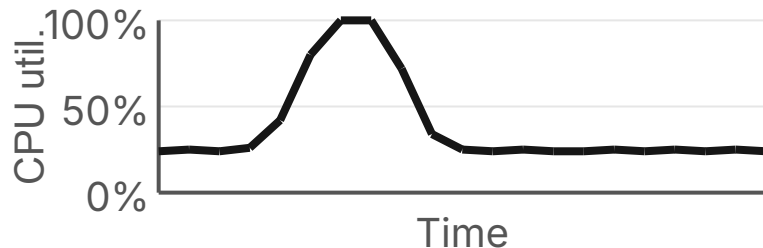
Merge workload B



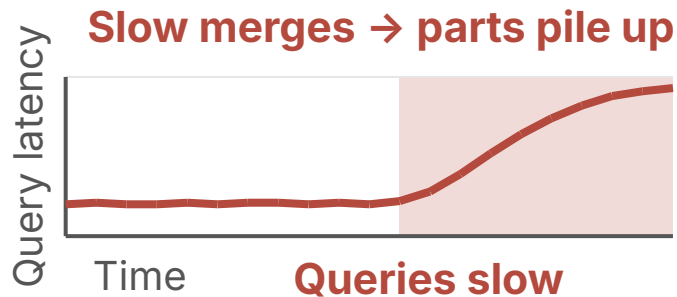
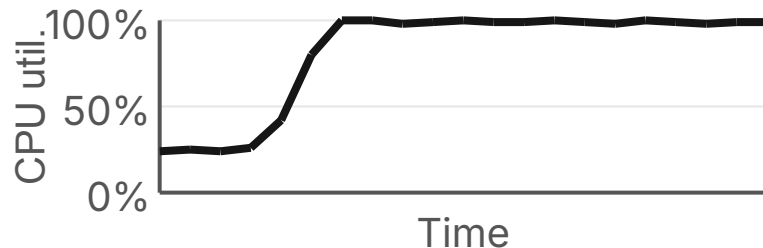
Same CPU signal. Queries still fast.

Faster merges do not always help queries

Merges finish quickly

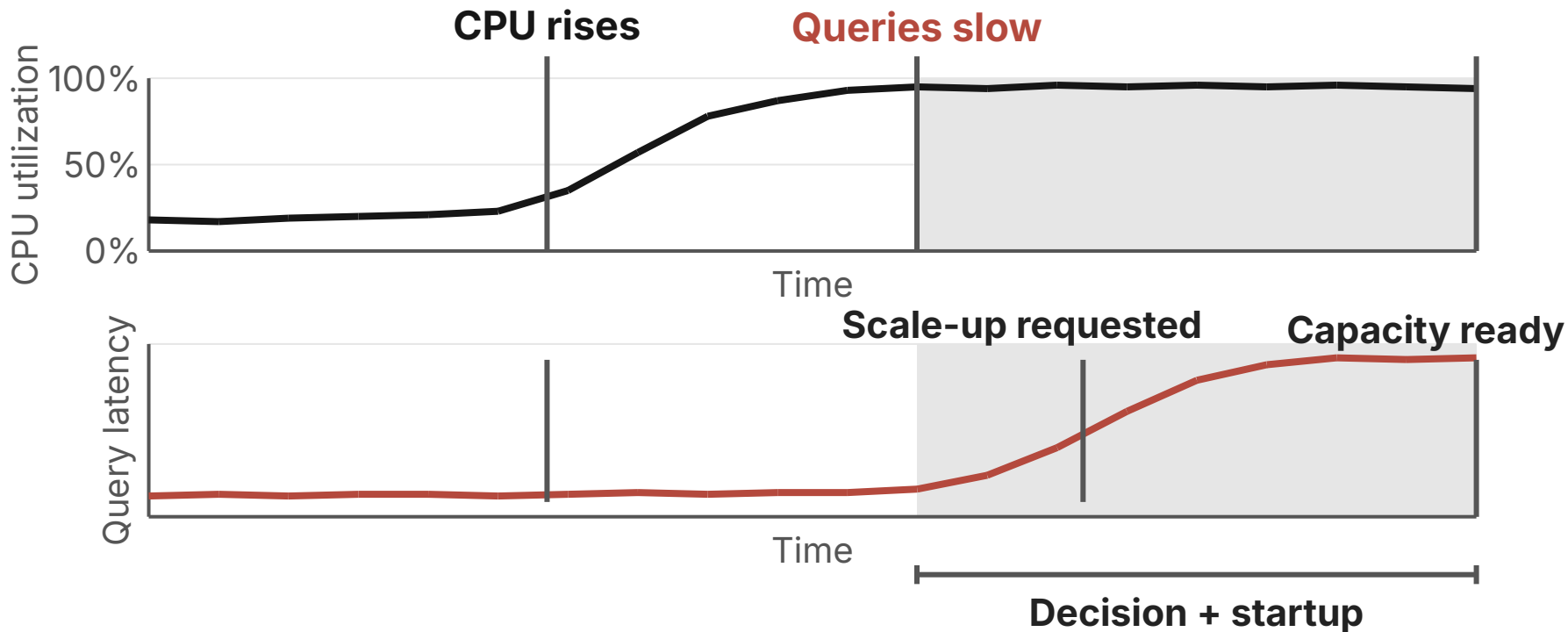


Merges fall behind



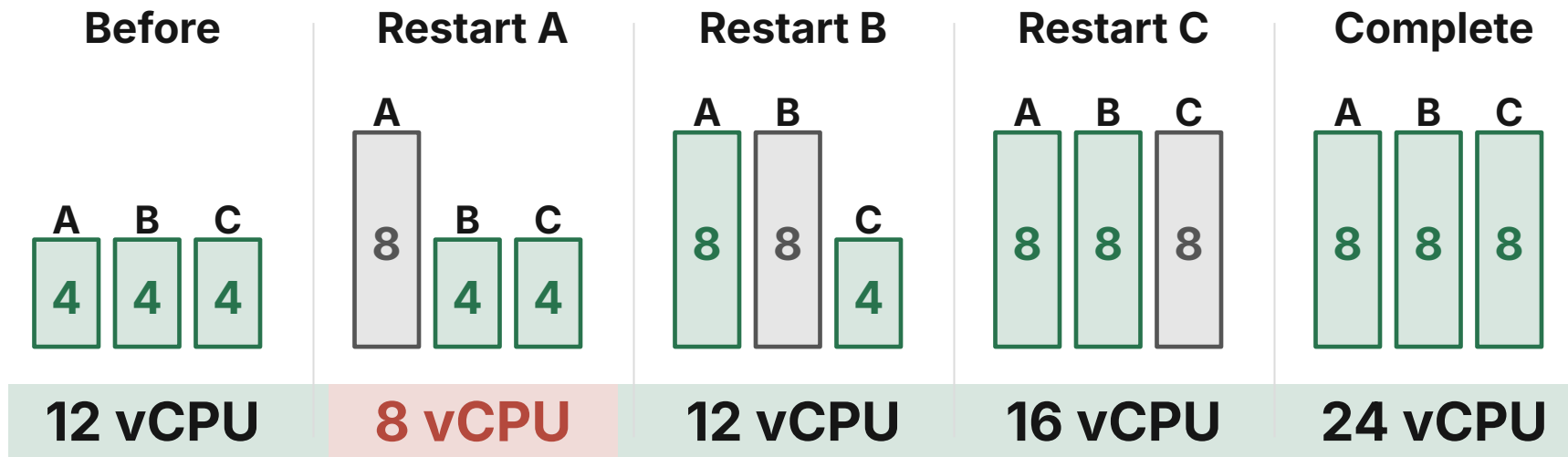
More compute speeds up both. Query impact differs.

Waiting for query latency can be too late



Scaling up first reduced serving capacity

12 → 24 vCPU, with a dip to 8 vCPU during replacement

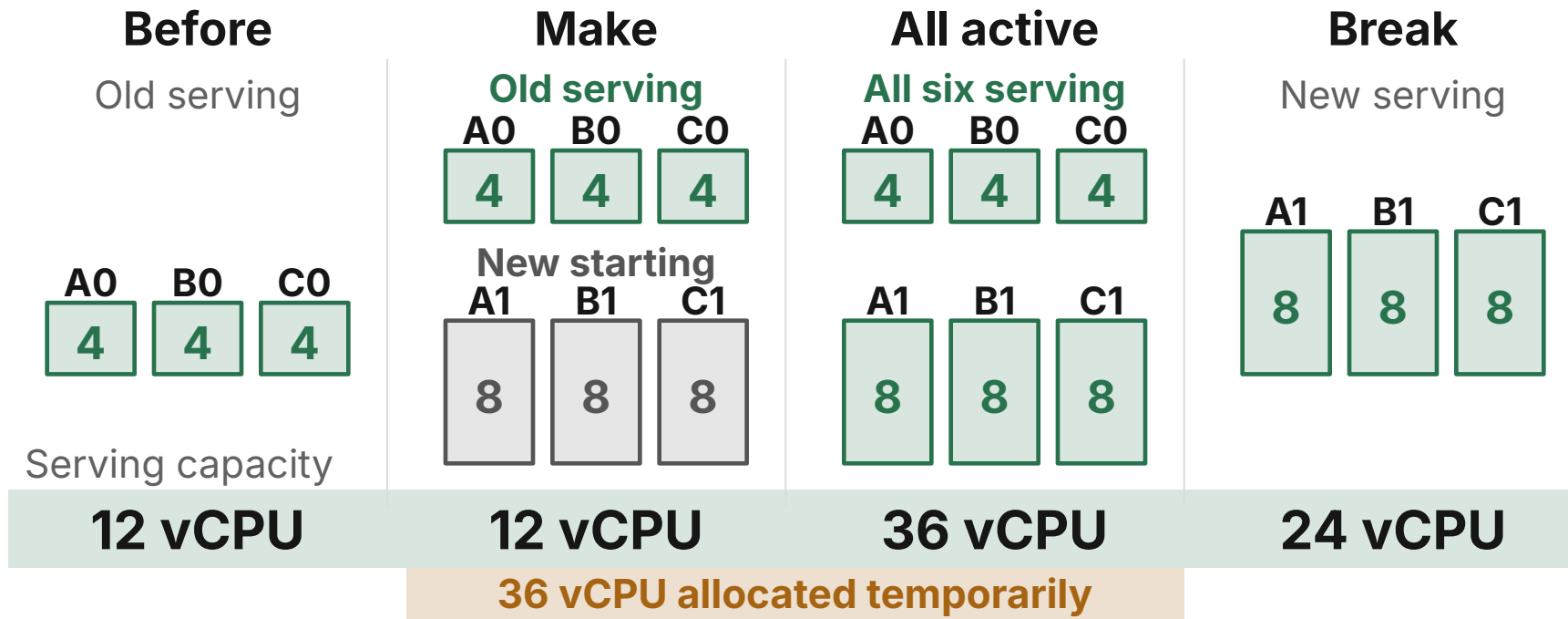


Serving capacity

Restarting: not serving

clickhouse.com/blog/make-before-break-faster-scaling-mechanics-for-clickhouse-cloud

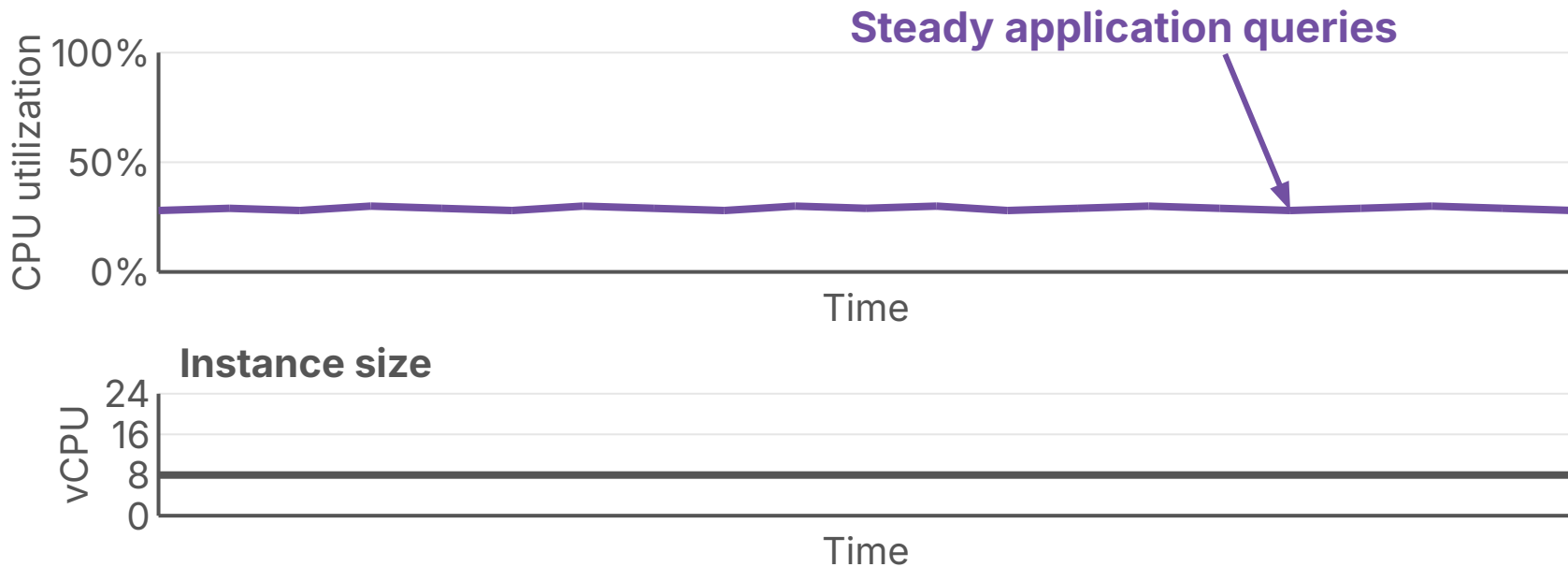
Make before break preserves serving capacity



clickhouse.com/blog/make-before-break-faster-scaling-mechanics-for-clickhouse-cloud

One cluster serves application queries

Dashboards, risk modeling, threat detection



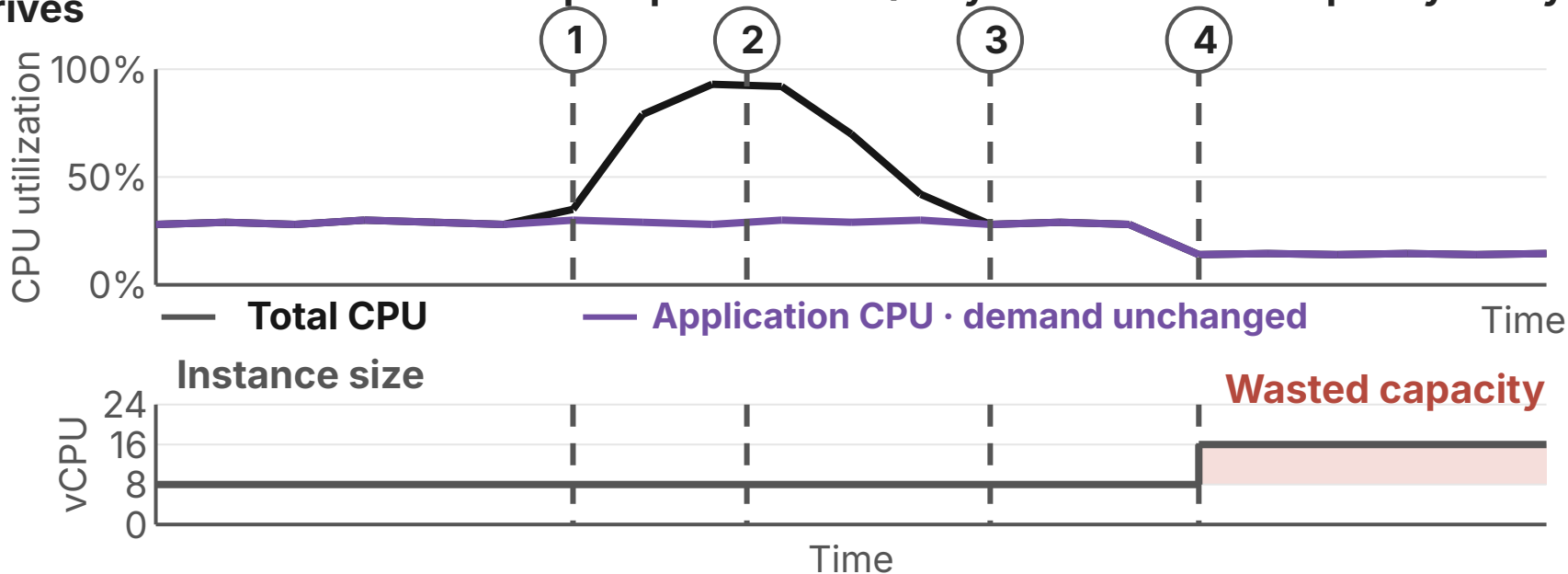
Batch queries trigger scale-up

1 Batch Query arrives

2 Scale-up request

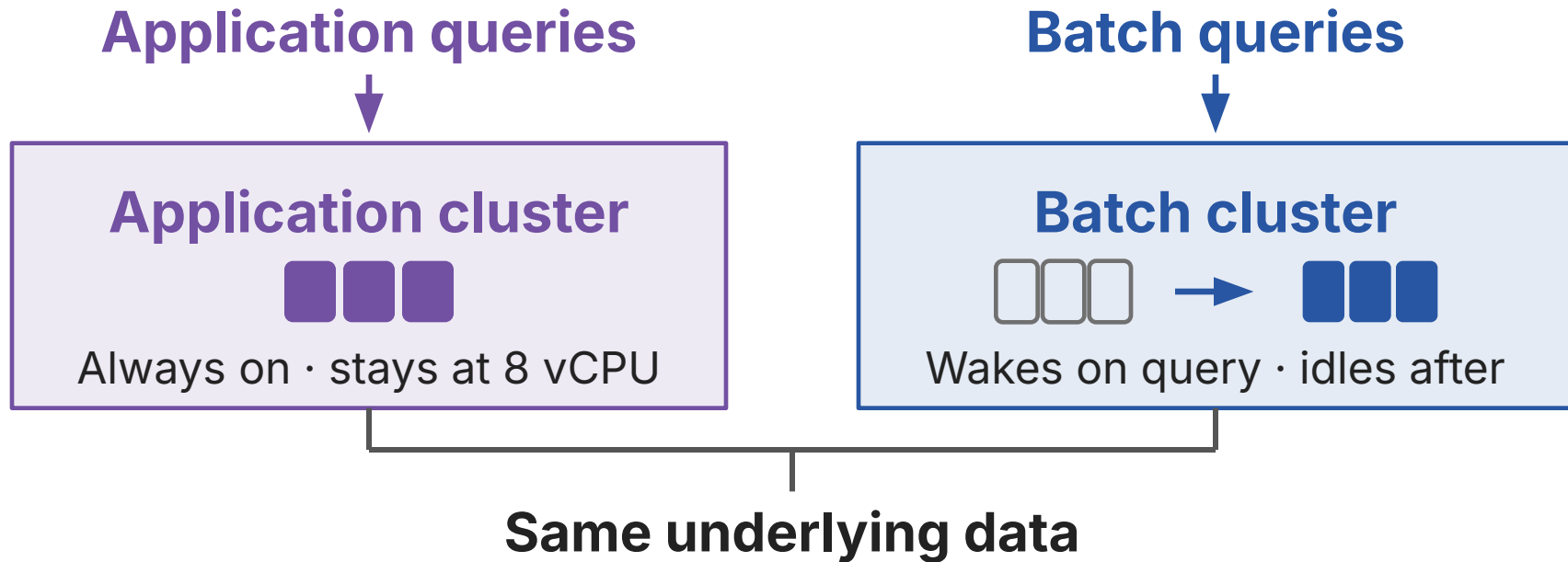
3 Query done

4 Capacity ready



The work is done. The extra capacity is wasted.

Separate clusters avoid overallocation



clickhouse.com/blog/introducing-warehouses-compute-compute-separation-in-clickhouse-cloud

Next: compute for the intermittent query

Application queries

Application cluster

Batch query opts in

Shared worker pool

Multi-tenant · automatic routing

SETTINGS · query opts in

```
make_distributed_plan = 1,  
distributed_plan_workers_num = 3
```

clickhouse.com/blog/on-demand-compute

Autoscaling is not just resource allocation

Performance benefit

Will accelerating this work improve query performance?

Scaling side effects

What happens to dependencies and serving capacity as we scale?

Include both in the autoscaling problem formulation.

Autoscaling is not just resource allocation

Performance benefit

Scaling side effects

Database Research Scientist

Overview

Application

Location

United States; Canada;
Germany; The
Netherlands; United
Kingdom

ClickHouse Labs is an applied research group that works closely with engineering teams to identify challenging problems in database systems and develop novel solutions. Our goal is to advance the state of the art and translate research ideas into real-world system improvements.

Core Software Engineer (C++) - Remote

Overview

Application

Location

The Netherlands

NOTE: this role can be located anywhere ClickHouse has a hiring presence.